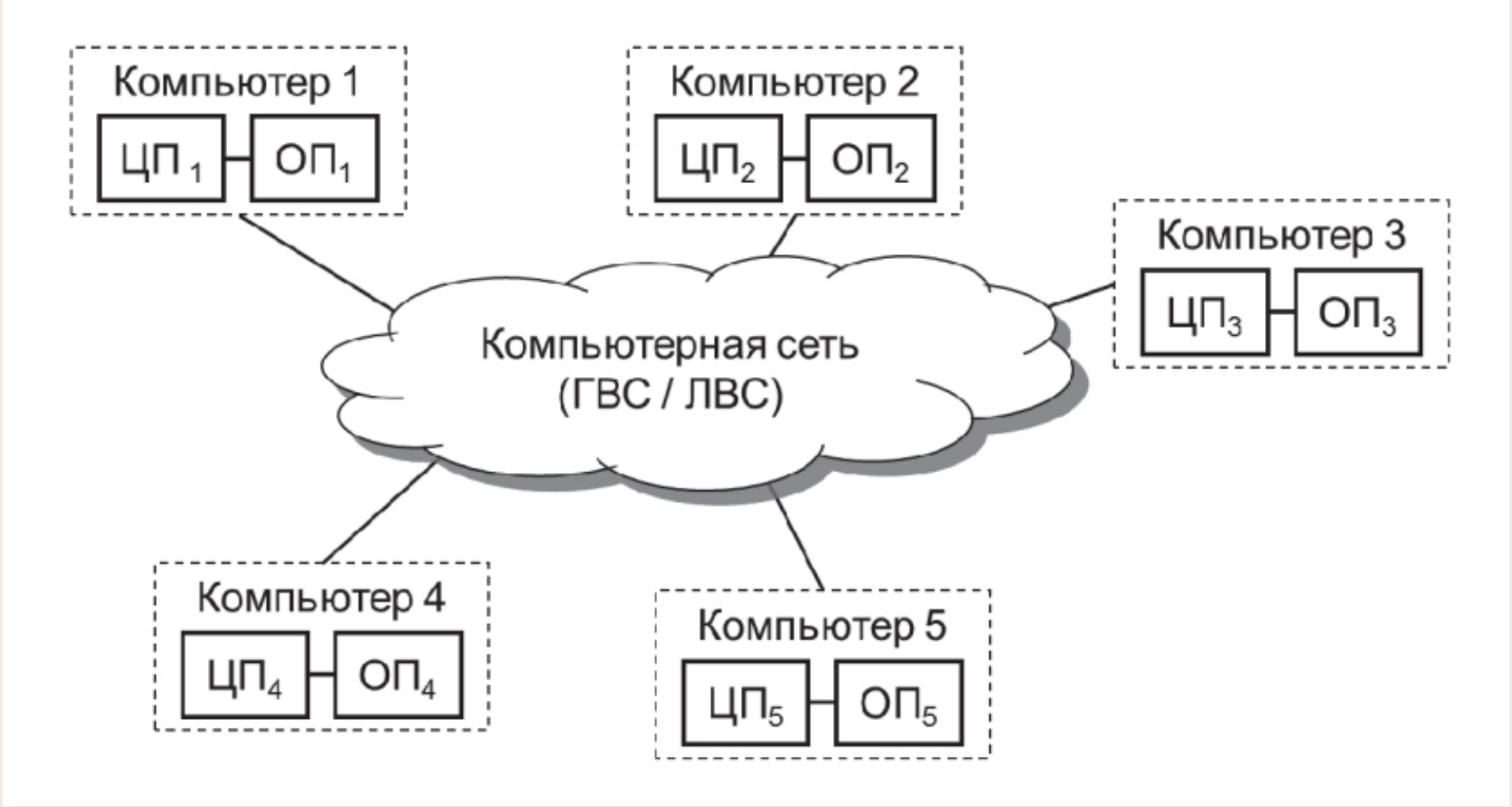
РАСПРЕДЕЛЕННЫЕ ВЫЧИСЛЕНИЯ И ПРИЛОЖЕНИЯ

Введение

Понятие распределенной системы

- Распределенная система – это такая система, в которой взаимодействие и синхронизация программных компонентов, выполняемых на независимых сетевых компьютерах, осуществляется посредством передачи сообщений.
- Распределенная система – набор независимых компьютеров, не имеющих общей совместно используемой памяти и общего единого времени (таймера) и взаимодействующих через коммуникационную сеть посредством передачи сообщений, где каждый компьютер использует свою собственную оперативную память и на котором выполняется отдельный экземпляр своей операционной системы. Однако эти операционные системы функционируют совместно, предоставляя свои службы друг другу для решения общей задачи.

Типичная распределенная система



Признаки распределенных систем

- Отсутствие единого времени для компонентов распределенной системы.
- Отсутствие общей памяти.
- Географическое распределение.
- Независимость и гетерогенность.
- Географически распределенная вычислительная среда.
- Требование увеличения производительности вычислений
- Совместное использование ресурсов.
- Отказоустойчивость.

Требования к распределенным системам

- **Прозрачность (Transparency):** способность скрывать свою распределенную природу, а именно, распределение процессов и ресурсов по множеству компьютеров, и представляться для пользователей и разработчиков приложений в виде единой централизованной компьютерной системы.
- **Открытость (Openness):** перенос, совместная работа
- **Масштабируемость**

Контейнеризация

- Популярная технология создания распределенных систем. Это подход к разработке программного обеспечения, при котором приложение или служба, их зависимости и конфигурация (абстрактные файлы манифеста развертывания) упаковываются вместе в образ контейнера.

https://habr.com/ru/articles/309556/

Оркестрация

- Оркестрация или оркестровка — если обратиться к wiki — это **автоматическое размещение, координация и управление сложными компьютерными системами и службами.**
- Такие системы, словно дирижеры оркестра, организуют расположение и координируют взаимосвязь инструментов в одном проекте, распределяют задачи между ними и контролируют их выполнение.
- Программы-оркестраторы управляют жизненными циклами контейнеров приложений. Но не только. Так как в разработке приложений все взаимосвязано, то системы оркестрации помогают еще автоматизировать различные производственные процессы, поэтому команды DevOps интегрируют их в CI/CD.

Docker

- Docker — это платформа, которая предназначена для разработки, развёртывания и запуска приложений в контейнерах. Слово «Docker» в последнее время стало чем-то вроде синонима слова «контейнеризация». И если вы ещё не пользуетесь Docker, но при этом работаете или собираетесь работать в сферах разработки приложений или анализа данных, то Docker — это то, с чем вы непременно встретитесь в будущем.
- Контейнеры Docker — это вызванные к жизни образы программ. Во время выполнения контейнера Docker внутри него обычно выполняется какая-то программа. Благодаря использованию Docker можно, на одном и том же компьютере, одновременно запускать множество контейнеров. И, как и любые другие программы, контейнеры Docker можно запускать, останавливать, удалять. Можно исследовать их содержимое и создавать их.

https://habr.com/ru/companies/ruvds/articles/438796/

Docker

- Образы контейнеров Docker можно сравнить с чертежами, с формочками для печенья, или с пресс-формами для изготовления пластиковых изделий. Образы — это неизменные шаблоны, которые используются для создания одинаковых контейнеров.
- В образе контейнера Docker содержится образ базовой операционной системы, код приложения, библиотеки, от которого оно зависит. Всё это скомпоновано в виде единой сущности, на основе которой можно создать контейнер.

Dockerfile

- Файл Dockerfile содержит набор инструкций, следуя которым Docker будет собирать образ контейнера. Этот файл содержит описание базового образа, который будет представлять собой исходный слой образа. Среди популярных официальных базовых образов можно отметить python, ubuntu, alpine.

```
FROM openjdk:17-oracle

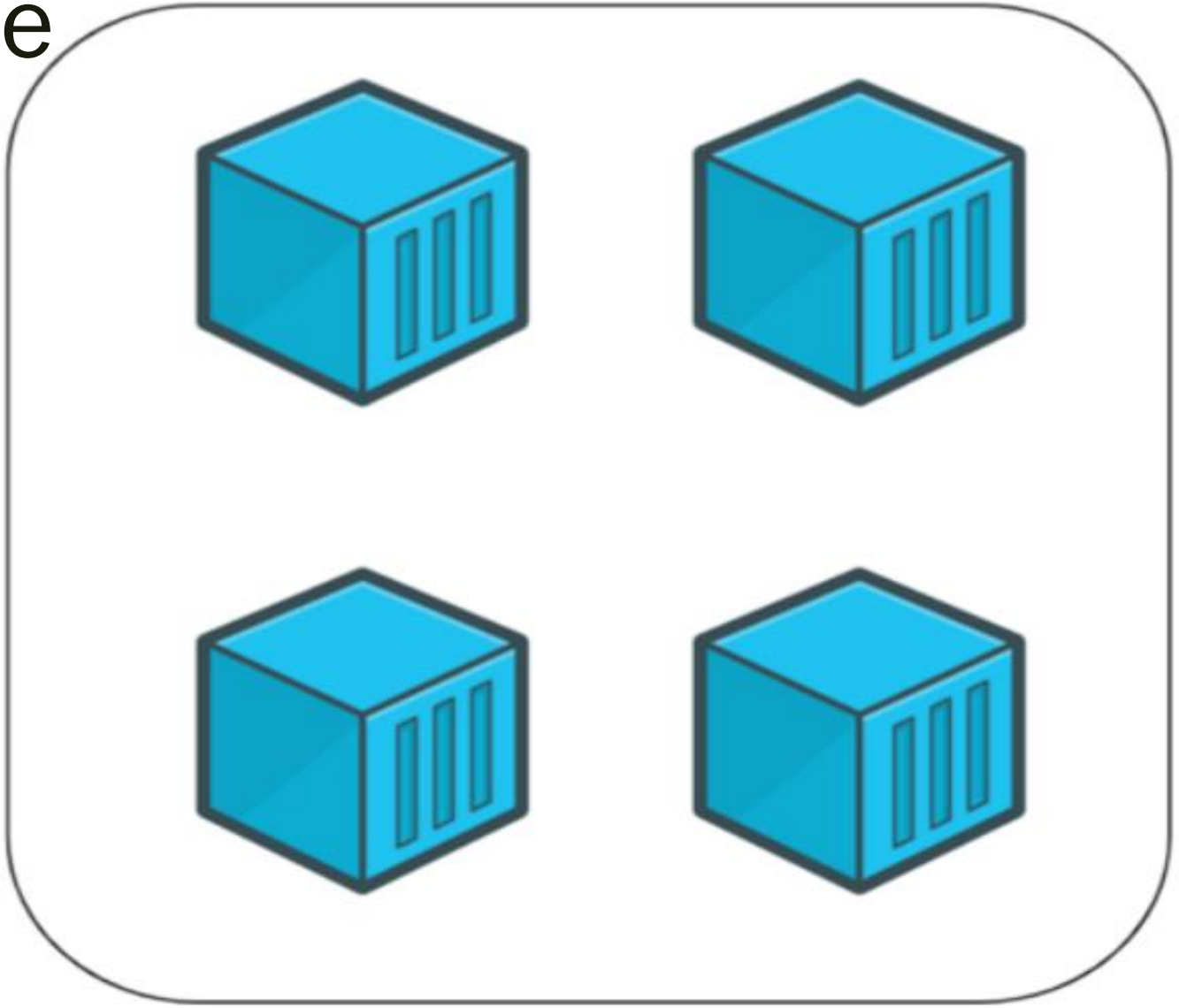
ADD back_app/target/test_app-1.0.0-SNAPSHOT.jar /app/
CMD ["java", "-Xmx200m", "-jar","-Dspring.profiles.active=dev", "/app/test_app-1.0.0-SNAPSHOT.jar"]

EXPOSE 8080
```

Docker compose



Docker



Docker-Compose

Технология Docker Compose, если описывать её упрощённо, позволяет, с помощью одной команды, запускать множество сервисов.

```yaml
version: '3.9'

services:
  db:
    image: postgres:14.1
    container_name: test-db
    restart: always
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: “testApplication"
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "postgres"
    networks:
       backend:
          aliases:
             - "db"
  back:
     build: ../testApplication
     profiles:
        -   dev
     ports:
        - "8080:8080"
     environment:
        db_url: jdbc:postgresql://db:5432/testAplication
        db_username: postgres
        db_password: "postgres"

     networks:
        backend:
           aliases:
              - "back"

networks:
   backend:
      driver: bridge
```

Системы управления миграциями базы данных

- Liquibase — это независимая от базы данных библиотека с открытым исходным кодом для отслеживания, управления и применения изменений схемы базы данных. Она была запущен в 2006 году, чтобы упростить отслеживание изменений в базе данных, особенно в гибкой среде разработки программного обеспечения.

Основные моменты работы

- Работает Liquibase так: **программист записывает все SQL-изменения базы данных в специальном Liquibase-формате** (xml, yaml, json, sql — на выбор). А библиотека **Liquibase запускает эти изменения. При этом Liquibase отслеживает саму себя — какие скрипты она уже запускала, а какие еще нет.** Для этого она создает в базе две служебные таблицы, в которых хранятся идентификаторы скриптов с датой запуска и т.п. И при запуске приложения Liquibase сама решает, какие скрипты выполнять, а какие нет.

Пример yaml

```yaml
databaseChangeLog:
- changeSet:
    id: 1596784098744-1
    author: myluc (generated)
    changes:
    - createTable:
        columns:
        - column:
            constraints:
              nullable: false
              primaryKey: true
              primaryKeyName: animal_pkey
            name: id
            type: BIGINT
        - column:
            name: name
            type: VARCHAR(255)
        tableName: animal
```

Пример SQL

```
--liquibase formatted sql
--changeset author:2-create-table-user

CREATE TABLE sale
(
id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
name varchar(255) NOT NULL
);
COMMENT ON TABLE user IS 'Table for user is system';
COMMENT ON COLUMN user.id IS 'id of user';
COMMENT ON COLUMN user.name IS ‘fio of user';
```

Пример. Spring Boot Application Развернем сервис и БД в одной докер-сети.